NETGUARD AI: A MACHINE LEARNING-BASED NETWORK INTRUSION DETECTION SYSTEM

A Mini Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Aditi Bhatnagar EN23CS301058

Abhinay Jat EN23CS301044

Under the Guidance of

Prof. Ashish Kumar Kumawat



Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

NETGUARD AI: A MACHINE LEARNING-BASED NETWORK INTRUSION DETECTION SYSTEM

A Mini Project Report

Submitted in partial fulfilment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Aditi Bhatnagar EN23CS301058

Abhinay Jat EN23CS301044

Under the Guidance of

Prof. Ashish Kumar Kumawat



Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work **"NETGUARD AI: A Machine Learning based Network Intrusion Detection System"** is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner
Prof Ashish Kumar Kumawat
Assistant Professor
Medicaps University

Declaration

We hereby declare that the project entitled "**NETGUARD AI: A Machine Learning based Network Intrusion Detection System"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering completed under the supervision of **Ashish Kumawat, Assistant Professor, Computer Science and Engineering,** Faculty of Engineering, Medi-Caps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Certificate

I, **Prof Ashish Kumawat** certify that the project entitled **"NETGUARD AI: A Machine Learning based Network Intrusion Detection System"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Aditi Bhatnagar & Abhinay Jat** is the record carried out by him/them under my guidance and that the work has not formed the basis of award of any other degree elsewhere.

_____

Prof Ashish Kumawat

Department of Computer Science and Engineering

Medicaps University, Indore

_____

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal,** who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik,** Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya ,** Associate Dean, Faculty of Computing Engineering, Medicaps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Aditi Bhatnagar

Abhinay Jat

B.Tech. III Year (A)

Department of Computer Science & Engineering

Faculty of Engineering

Medi-Caps University, Indore

Abstract

Modern computer networks are under constant and evolving threat from sophisticated cyber attacks including Denial-of-Service (DoS), port scanning, backdoor exploitation, shellcode injection, and data exfiltration. Traditional signature-based Intrusion Detection Systems (IDS) fail to detect zero-day and polymorphic threats, creating an urgent need for intelligent, learning-based approaches to network security.

NetGuard AI is a machine learning-powered Network Intrusion Detection System (NIDS) designed to analyze network traffic captured in PCAP format and automatically classify flows as either normal or malicious. The system extracts 32 flow-level features from raw packet captures using a dual-engine pipeline combining NFStream for statistical flow analysis and Scapy for deep packet inspection. These features cover packet timing, byte counts, TCP window sizes, round-trip times, TTL values, HTTP transaction depth, FTP login signals, and a suite of connection-tracking counters.

The prediction engine employs an ensemble of three supervised classifiers — Random Forest, XGBoost, and LightGBM — trained on the UNSW-NB15 benchmark dataset containing over 2.5 million network flows spanning nine attack categories. Simultaneously, an Isolation Forest performs unsupervised anomaly detection, capturing behavioral deviations that supervised models may miss. The outputs of both sub-systems are fused using a Bayesian-inspired 50/50 weighted combination, producing a unified threat score per flow. A configurable threshold mechanism converts these scores into binary ATTACK or NORMAL verdicts, with an overall session verdict triggered when attack ratio exceeds 30% of analyzed flows.

The user interface is a Streamlit-based dashboard titled NetGuard AI, offering drag-and-drop PCAP file upload, multi-step animated analysis progress, and rich result visualizations including per-flow threat score charts, risk level indicators, network status flags, and analyst notes. A live capture module using Scapy's AsyncSniffer supports real-time packet recording from network interfaces for on-the-fly analysis.

Keywords: Network Intrusion Detection System, Machine Learning, Ensemble Learning, Isolation Forest, XGBoost, LightGBM, Random Forest, Bayesian Fusion, UNSW-NB15, PCAP Analysis, NFStream, Scapy, Anomaly Detection, Cybersecurity, Threat Detection

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of figures	viii
	List of tables	ix
	Abbreviations	x
	Notations & Symbols	xi
Chapter 1	Introduction (Whichever is applicable)	
	1.1 Introduction	1
	1.2 Literature Review	
	1.3 Objectives	
	1.4 Significance	
	1.5 Research Design	
	1.6 Source of Data	
	1.7 Chapter Scheme	
Chapter 2	REQUIREMENTS SPECIFICATION	
	2.1 User Characteristics	
	2.2 Functional Requirements	
	2.3 Dependencies	
	2.4 Performance Requirements	
	2.5 Hardware Requirements	
	2.6 Constraints & Assumptions	
Chapter 3	DESIGN (Whichever is applicable)	
	3.1 Algorithm (if Applicable)	
	3.2 Function Oriented Design for procedural approach	

	3.3 **System Design (**Whichever is applicable)	
	3.3.1 Data Flow Diagrams (Level 0,Level1)	
	3.3.2 Activity Diagram	
	3.3.3 Flow Chart	
	3.3.4 Class Diagram	
	3.3.5 ER Diagram	
	3.3.6 Sequence diagram	
	3.4 **Database Design**	
	3.4.1 Logical Database Design	
	3.4.2 Physical Database Design	
Chapter 4	Implementation, Testing, and Maintenance	
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	
	4.2 Testing Techniques and Test Plans (According to project)	
	4.3 Installation Instructions	
	4.4 End User Instructions	
Chapter 5	Results and Discussions	
	5.1 User Interface Representation (of Respective Project)	
	5.2 Brief Description of Various Modules of the system	
	5.3 Snapshots of system with brief detail of each	
	5.4 Back Ends Representation (Database to be used)	
	5.5 Snapshots of Database Tables with brief description	
Chapter 6	Summary and Conclusions	
Chapter 7	Future scope	
	Appendix	
	Bibliography	
	List of Publications (If any)	
	Reprints of publications(If any)	

List of Figures

Figure No.	Title	Page No.
Fig. 3.1	Data Flow Diagram Level 0 (Context Diagram)	14
Fig. 3.2	Data Flow Diagram Level 1	15
Fig. 3.3	Activity Diagram of NetGuard AI NIDS	16
Fig. 3.4	Flowchart of ML Prediction Pipeline	17
Fig. 3.5	Class Diagram	18
Fig. 3.6	Sequence Diagram	19
Fig. 5.1	NetGuard AI Dashboard – PCAP Upload Interface	26
Fig. 5.2	Threat Score Chart (Rolling Average)	28
Fig. 5.3	Session Safety Score and Risk Level Display	29

List of Tables

Table No.	Title	Page No.
Table 2.1	Functional Requirements of NetGuard AI NIDS	9
Table 2.2	Hardware and Software Requirements	11
Table 3.1	32 Flow-Level Features Extracted from PCAP	13
Table 4.1	Technologies and Libraries Used	21
Table 4.2	Test Cases – PCAP Analysis Module	23

Abbreviations

Abbreviation	Full Form
NIDS	Network Intrusion Detection System
ML	Machine Learning
RF	Random Forest
XGB	XGBoost (Extreme Gradient Boosting)
LGBM	LightGBM (Light Gradient Boosting Machine)
IF	Isolation Forest
PCAP	Packet Capture
DoS	Denial of Service
HTTP	HyperText Transfer Protocol
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
TTL	Time to Live
RTT	Round Trip Time
FTP	File Transfer Protocol

Notations & Symbols

Notation/Symbol	Description
P(attack)	Probability of attack output by ensemble classifiers
S_anomaly	Anomaly score from Isolation Forest
S_combined	Bayesian combined threat score (0 to 1)
w_e	Weight assigned to ensemble classifier score (0.5)
w_a	Weight assigned to anomaly detection score (0.5)
theta	Decision threshold for classification
f_i	i-th feature in the 32-dimensional feature vector
n	Number of network flows analyzed

Chapter - 1

Introduction

1.1 Introduction

The proliferation of networked computing systems across industries — from financial services and healthcare to government and critical infrastructure — has made network security a matter of paramount importance. Cyber-attacks have grown both in frequency and sophistication, with adversaries employing advanced techniques such as polymorphic malware, encrypted command-and-control channels, and multi-vector attack campaigns. Traditional perimeter-based defenses such as firewalls and antivirus software, which rely on static signature databases, are increasingly inadequate against novel, zero-day threats.

Network Intrusion Detection Systems (NIDS) represent a proactive approach to security monitoring by analyzing network traffic in real-time or post-hoc to identify malicious activity. However, the effectiveness of NIDS depends critically on the detection methodology employed. Rule-based and signature-based systems suffer from high false-negative rates for new attack variants and require constant manual signature updates. Machine learning (ML)-based NIDS address these limitations by learning complex decision boundaries from historical traffic data, enabling generalization to previously unseen attack patterns.

This project presents NetGuard AI, an ML-based NIDS that combines the strengths of supervised ensemble learning with unsupervised anomaly detection. The system analyzes PCAP (Packet Capture) files, extracts a rich set of 32 flow-level features using NFStream and Scapy, and applies a Bayesian combination of ensemble classifier predictions and Isolation Forest anomaly scores to produce a robust threat assessment. The results are presented through an interactive web dashboard built with Streamlit.

1.2 Literature Review

A substantial body of research exists on the application of machine learning to network intrusion detection. Tavallaee et al. (2009) identified significant issues with the original KDD Cup 1999 dataset and proposed the NSL-KDD dataset, which addressed redundancy and imbalance issues. Moustafa and Slay (2015) introduced the UNSW-NB15 dataset, which NetGuard AI employs, as a more representative benchmark capturing nine modern attack categories.

Ensemble learning methods have demonstrated superior performance in intrusion detection tasks. Breiman (2001) introduced Random Forest, an ensemble of decision trees using bagging and feature randomization, which has proven highly effective for high-dimensional classification tasks. Chen and Guestrin (2016) proposed XGBoost, which employs gradient boosting with regularization to achieve state-of-the-art performance across multiple domains. Ke et al. (2017) introduced LightGBM, a gradient boosting framework with histogram-based algorithms that significantly reduces training time while maintaining accuracy.

Anomaly detection using Isolation Forest, proposed by Liu et al. (2008), provides an unsupervised approach that isolates anomalies by randomly partitioning feature space. Unlike density-based methods, Isolation Forest scales efficiently to high-dimensional data and does not require a normal traffic profile to be explicitly modeled. The combination of supervised and unsupervised approaches in a hybrid system has been shown by multiple studies (Bhuyan et al., 2014) to improve overall detection performance, particularly for low-volume and novel attacks.

Recent work on feature extraction from network flows has highlighted the importance of using both coarse flow statistics and fine-grained packet-level attributes. NFStream, introduced by Aouini and Pekar (2021), provides an efficient framework for flow-level feature extraction from PCAP files, while Scapy enables deep packet inspection for extracting raw packet attributes such as TTL, TCP window size, and payload length. The integration of both tools in a unified feature extraction pipeline is a distinguishing characteristic of NetGuard AI.

1.3 Objectives

The primary objectives of the NetGuard AI project are as follows:

- To design and implement a machine learning-based Network Intrusion Detection System capable of detecting diverse attack types from network traffic captured in PCAP format.
- To develop a feature extraction pipeline using NFStream and Scapy that extracts 32 meaningful flow-level and packet-level features from raw PCAP files.
- To train and evaluate three ensemble classifiers — Random Forest, XGBoost, and LightGBM — on the UNSW-NB15 dataset and combine their outputs for improved detection accuracy.
- To incorporate an Isolation Forest anomaly detection model for unsupervised threat scoring to capture novel attacks not well-represented in training data.
- To implement a Bayesian fusion mechanism that combines supervised classification and unsupervised anomaly scores into a unified threat score.

- To implement a configurable threshold-based verdict system that classifies traffic sessions as NORMAL or ATTACK with associated risk levels and analyst notes.
- To develop a user-friendly Streamlit dashboard enabling upload-based PCAP analysis and live network capture with real-time threat visualization.
- To produce human-readable security verdicts including risk level, network status, threat intensity, and traffic behavior classification.

1.4 Significance

NetGuard AI demonstrates how contemporary machine learning techniques can be applied to create a practical, deployable network security tool. Its significance spans several dimensions:

- Academic Contribution: Applies Bayesian fusion of heterogeneous ML models (supervised + unsupervised) to NIDS, contributing to the growing body of ensemble-based security research.
- Practical Utility: Provides network administrators and security analysts with a tool that requires no signature updates, automatically adapts to new traffic patterns through retraining, and delivers human-readable verdicts rather than raw anomaly scores.
- Educational Value: Serves as a comprehensive demonstration of the end-to-end ML pipeline from raw packet capture through feature engineering, model training, score fusion, and interactive visualization.
- Dataset Coverage: By training on UNSW-NB15, the system addresses nine realistic attack categories including DoS, Fuzzing, Backdoors, Exploits, Reconnaissance, Shellcode, Worms, Analysis, and Generic attacks.

1.5 Research Design

The project follows a structured software engineering methodology incorporating requirements analysis, system design, iterative implementation, and testing phases. The research design involves the following key components: dataset-based model training using UNSW-NB15, empirical feature selection to identify the most discriminative 32 features, cross-validation for model evaluation, and integration testing of the end-to-end pipeline with real PCAP files. The system design follows a three-tier architecture: data processing (feature extraction), inference (ML pipeline), and presentation (dashboard). The research follows an applied engineering methodology

organized into five phases: (1) Dataset acquisition and exploratory analysis of UNSW-NB15; (2) Feature selection and extraction pipeline development; (3) Model training, hyperparameter tuning, and threshold calibration; (4) System integration and dashboard development; and (5) Evaluation and documentation. The prediction pipeline is fully modular, enabling independent replacement or upgrading of any component.

1.6 Source of Data

The primary dataset used for training and evaluation is UNSW-NB15, created by the Australian Centre for Cyber Security (ACCS) at UNSW Canberra. It contains approximately 2.5 million records with 49 features spanning nine attack categories: Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. For live system testing, real PCAP files were used that include both benign traffic and traffic from publicly available attack simulation tools.

1.7 Chapter Scheme

Chapter 1 provides the introduction, literature review, and objectives. Chapter 2 presents the requirements specification covering functional requirements, dependencies, and hardware/software needs. Chapter 3 covers the system design including data flow diagrams, activity diagrams, flowcharts, and class structures. Chapter 4 details implementation, testing, and installation instructions. Chapter 5 presents results and discussions with dashboard screenshots and module descriptions. Chapter 6 summarizes conclusions, and Chapter 7 outlines future scope.

Chapter - 2

Requirements Specification

2.1 User Characteristics

The primary users of NetGuard AI are network security analysts, system administrators, and computer science students studying cybersecurity. Users are expected to possess basic familiarity with network traffic concepts and be capable of generating PCAP files using tools such as Wireshark or tcpdump. The dashboard is designed to be accessible to non-ML experts through plain-English result summaries and visual indicators, while providing sufficient technical depth for advanced users.

2.2 Functional Requirements

Table 2.1 summarizes the functional requirements of NetGuard AI NIDS:

ID	Requirement	Description
FR-01	PCAP File Upload	System shall accept .pcap, .pcapng, and .cap files via drag-and-drop.
FR-02	Feature Extraction	System shall extract 32 flow-level features per network flow using NFStream + Scapy.
FR-03	ML Prediction	System shall run RF, XGBoost, and LightGBM ensemble with Isolation Forest anomaly scoring.
FR-04	Bayesian Fusion	System shall combine ensemble and anomaly scores with 50/50 weighting.
FR-05	Verdict Generation	System shall classify session as NORMAL or ATTACK with risk level and analyst note.
FR-06	Dashboard Display	System shall display threat score chart, metrics panel, and session history.

FR-07	Live Capture	System shall capture live packets from a specified network interface using AsyncSniffer.
FR-08	Progress Feedback	System shall display multi-step animated progress during analysis.

2.3 Dependencies

The system relies on the following external dependencies:

Library	Version	Purpose
streamlit	≥1.30	Web UI framework
nfstream	≥6.5	Flow-level statistical feature extraction
scapy	≥2.5	Packet-level deep inspection (TTL, TCP flags, payload)
scikit-learn	≥1.3	Random Forest, Isolation Forest, preprocessing
xgboost	≥2.0	Extreme Gradient Boosting classifier
lightgbm	≥4.0	Light Gradient Boosting Machine classifier
pandas	≥2.0	Tabular data handling
numpy	≥1.24	Numerical computation
plotly	≥5.18	Interactive threat visualization charts
joblib	≥1.3	Model serialization / deserialization

2.4 Performance Requirements

The system is expected to meet the following performance requirements to be operationally useful:

- Feature extraction from a 10 MB PCAP file should complete within 30 seconds on a standard modern CPU.

- Ensemble inference for up to 10,000 flows should complete within 5 seconds.
- Dashboard should remain responsive with a Streamlit session refresh cycle under 2 seconds.
- Models are loaded once at startup and cached in session state to eliminate per-request reload overhead.

2.5 Hardware Requirements

Component	Minimum Specification
Processor	Dual-core 2.0 GHz (Quad-core recommended)
RAM	4 GB (8 GB recommended for large PCAP files)
Storage	2 GB free disk space for models and temporary files
Operating System	Linux (Ubuntu 20.04+), macOS 12+, Windows 10+
Network Interface	Any Ethernet or Wi-Fi adapter (for live capture mode)
Python	3.10 or above

2.6 Constraints & Assumptions

- Live capture requires administrator/root privileges and libpcap (Linux/macOS) or Npcap (Windows).
- The system assumes input PCAP files contain IP-level traffic; non-IP protocols are ignored during feature extraction.
- Model artifacts (classifiers, scalers, encoders, feature orders, threshold) must be pre-trained and placed in the models/ directory before running the system.
- The system targets binary classification (ATTACK vs NORMAL) and does not perform multi-class attack categorization in this version.
- Internet connectivity is required only for fetching Google Fonts used in the dashboard CSS; all ML inference is performed locally.

Chapter - 3

Design

3.1 Algorithm

The core ML pipeline of NetGuard AI follows a multi-stage algorithmic design. Table 3.1 below presents the 32 flow-level features extracted as input to the ML models:

#	Feature	#	Feature
1	dur	17	ct_srv_src
2	proto	18	ct_state_ttl
3	service	19	ct_src_dport_ltm
4	state	20	ct_dst_sport_ltm
5	spkts	21	ct_dst_src_ltm
6	dpkts	22	is_ftp_login
7	sbytes	23	ct_flw_http_mthd
8	dbytes	24	ct_srv_dst
9	sttl	25	is_sm_ips_ports
10	dttl	26	sload
11	sinpkt	27	dload
12	swin	28	sloss
13	dwin	29	dloss
14	tcprtt	30	smean
15	synack	31	dmean
16	trans_depth	32	response_body_len

The prediction algorithm proceeds as follows:

- Stage 1 — PCAP Ingestion: Raw packet capture file is loaded via NFStream to produce a flow-level DataFrame containing bidirectional statistics.

- Stage 2 — Basic Feature Extraction: NFStream statistics are transformed into 18 primary features including duration, packet counts, byte counts, load metrics, inter-packet timing, protocol, service, and state.
- Stage 3 — Deep Packet Inspection: A single sequential PCAP pass using Scapy collects packet-level data including TTL values, TCP window sizes, SYN/SYN-ACK/ACK timestamps for RTT computation, HTTP transaction depth, response body lengths, and FTP login signals.
- Stage 4 — Connection Tracking: A sliding window of 100 recent flows computes six connection-tracking counters capturing temporal relationships between source/destination IPs, ports, services, states, and TTL groups.
- Stage 5 — Preprocessing: Categorical features (proto, service, state) are label-encoded using pre-fitted encoders. Remaining features are standardized using MinMaxScaler. The 32-feature DataFrame is assembled in the exact order expected by each model.
- Stage 6 — Dual Scoring: The ensemble of RF + LightGBM + XGBoost produces averaged attack probabilities. Simultaneously, Isolation Forest scores each flow for behavioral anomaly via its decision function, normalized to [0,1].
- Stage 7 — Bayesian Fusion and Verdict: Ensemble and Isolation Forest scores are combined (50/50 weighting). Combined scores are compared to a learned threshold; sessions with ≥30% attack-classified flows are labeled ATTACK.

3.2 Function Oriented Design

The system is organized into two main Python modules with clearly defined functional interfaces:

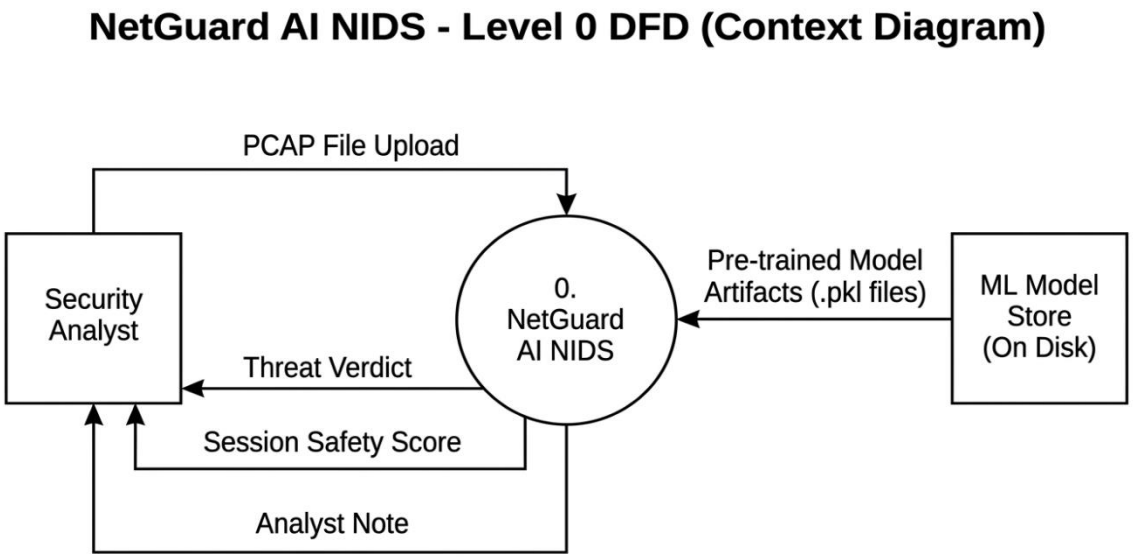
- feature_extraction.py: Contains load_pcap(), extract_basic_features(), _single_pcap_pass(), apply_pcap_features(), compute_ct_features(), compute_ct_flw_http_mthd(), and the master function extract_all_features().
- prediction.py: Contains load_artifacts(), preprocess(), run_isolation_forest(), run_ensemble(), combine_and_decide(), and the master function run_prediction().
- dashboard.py: Contains render_topbar(), render_results(), render_live_capture_tab(), run_full_analysis(), and the main() entry point.

3.3 System Design

3.2.1 Data Flow Diagram (Level 0 and Level 1)

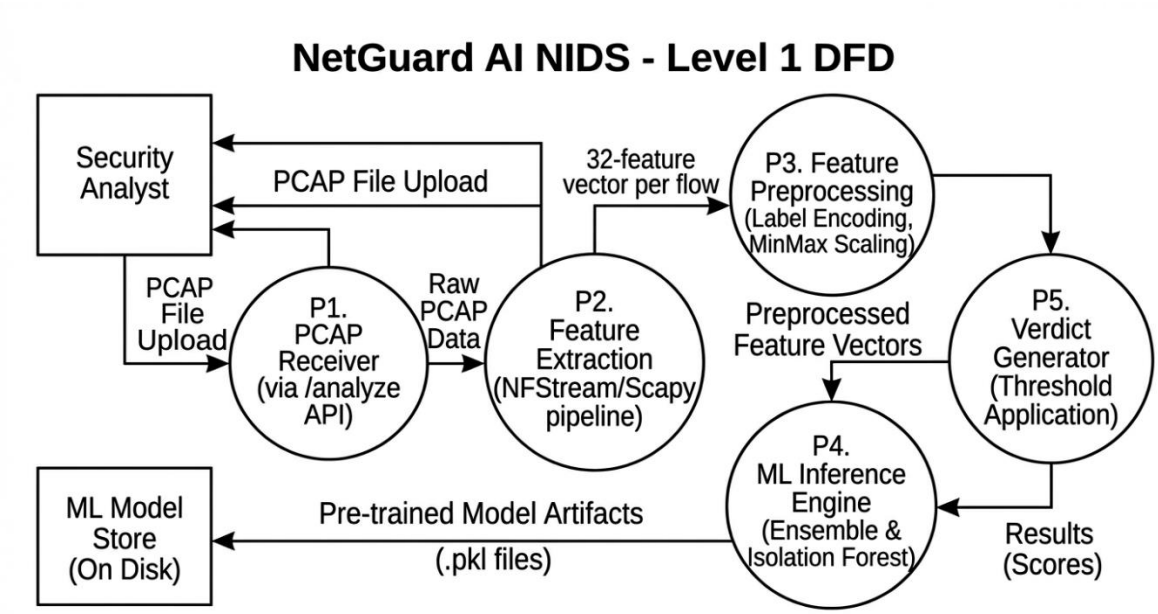
Level 0 DFD (Context Diagram):

At the highest level of abstraction, NetGuard AI NIDS is a single process that accepts PCAP files from the Security Analyst and returns a Threat Verdict. The external entities are: (1) Security Analyst — uploads PCAP files and views analysis results; (2) ML Model Store — pre-trained model artifacts (.pkl files) stored on disk. The system outputs the threat verdict, session safety score, and analyst note to the Security Analyst.



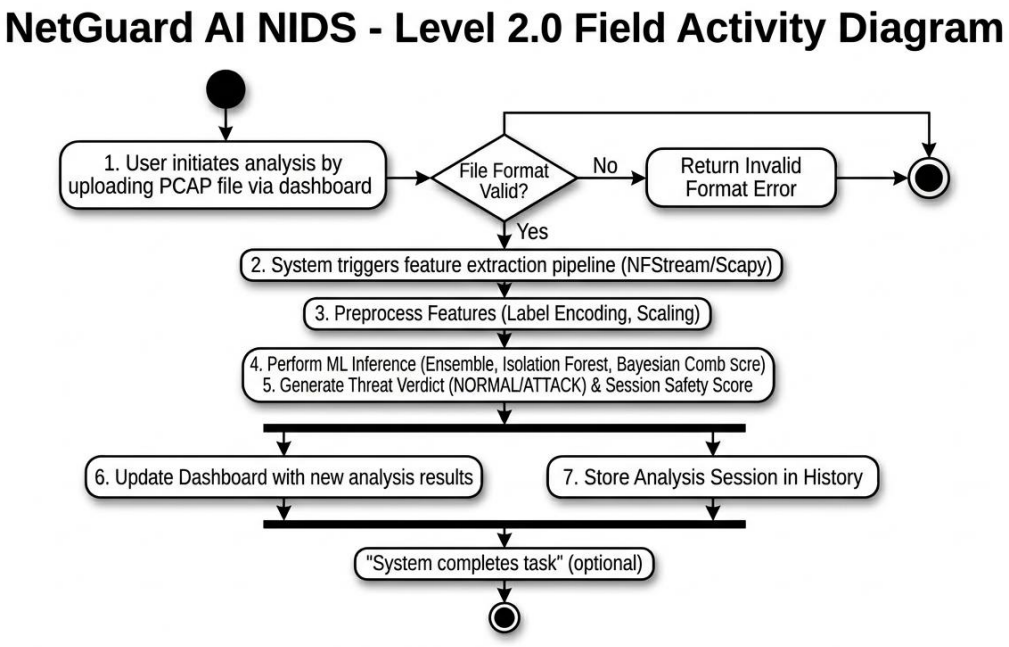
Level 1 DFD:

At the first level of decomposition, the system is broken into five major processes: (P1) PCAP Receiver — receives uploaded file via the /analyze API endpoint; (P2) Feature Extraction — NFStream and Scapy pipeline that transforms raw packets into a 32-feature vector per flow; (P3) Feature Preprocessing — label encoding and MinMax scaling of feature vectors; (P4) ML Inference Engine — applies ensemble classifiers and Isolation Forest, computes Bayesian combined score; (P5) Verdict Generator — applies threshold to produce NORMAL/ATTACK verdicts and generates human-readable analysis output.



3.2.2 Activity Diagram

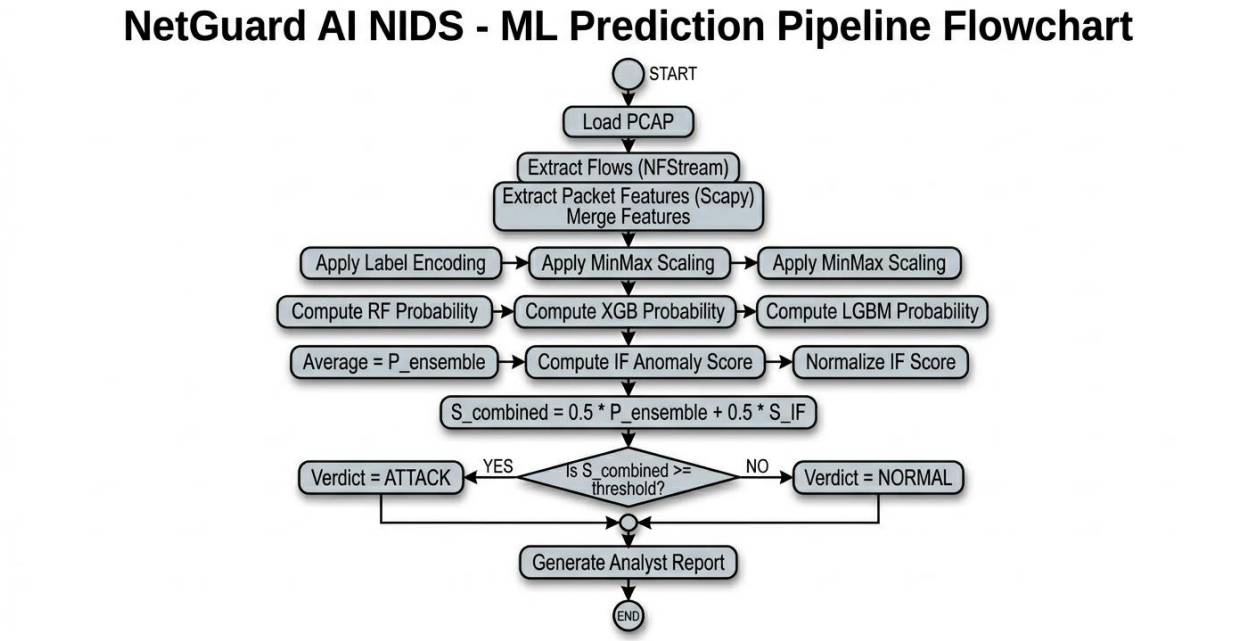
The activity diagram illustrates the control flow of a single PCAP analysis request. The user initiates the process by uploading a PCAP file through the dashboard. The system validates the file format. If invalid, an error is returned. If valid, the feature extraction pipeline is triggered, followed by preprocessing, ML inference, and verdict generation. The dashboard is updated with the new analysis results, and the session is stored in history. The process terminates when the dashboard displays the final results.



3.2.3 Flowchart

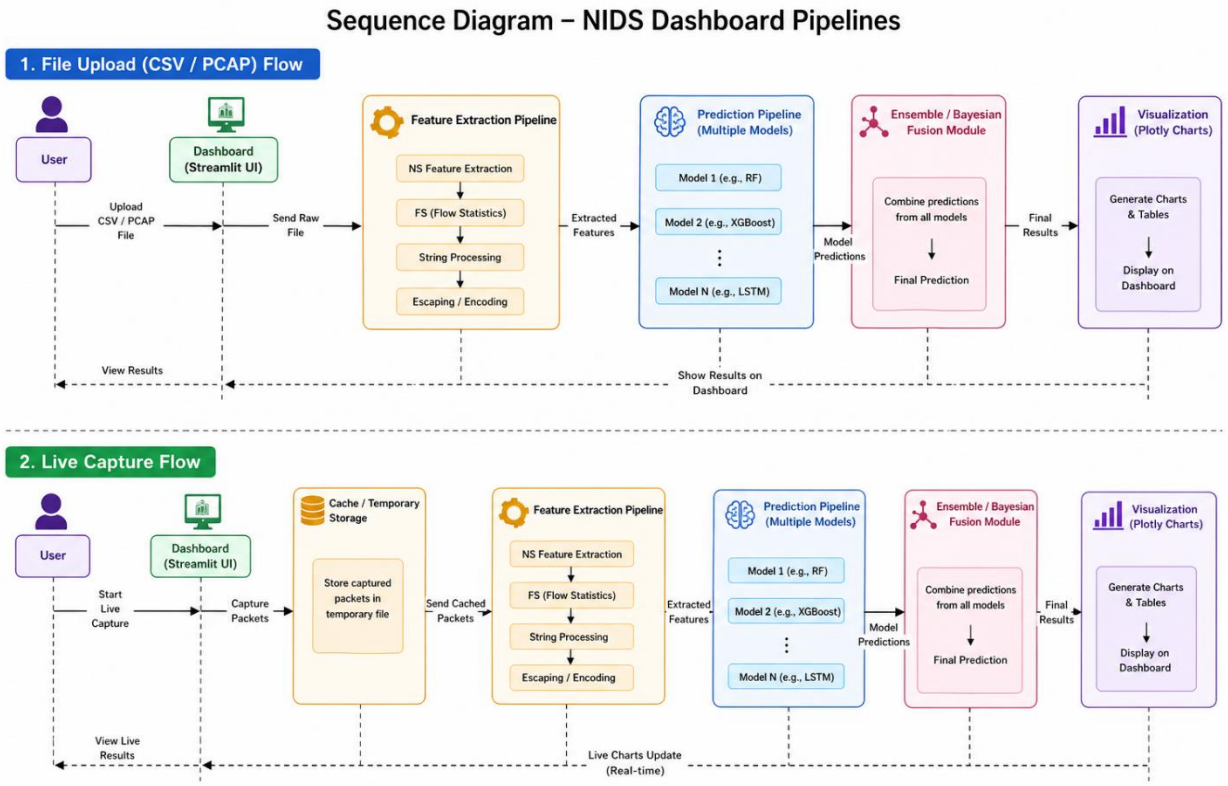
The ML Prediction Pipeline flowchart follows this sequence:

START → Load PCAP → Extract Flows (NFStream) → Extract Packet Features (Scapy) → Merge Features → Apply Label Encoding → Apply MinMax Scaling → Compute RF Probability → Compute XGB Probability → Compute LGBM Probability → Average = P_ensemble → Compute IF Anomaly Score → Normalize IF Score → S_combined = 0.5 * P_ensemble + 0.5 * S_IF → Is S_combined >= threshold? → YES: Verdict = ATTACK → NO: Verdict = NORMAL → Generate Analyst Report → END.



3.2.5 Sequence Diagram

The sequence diagram shows interactions between: User, DashboardUI (Flask), BackendAPI (FastAPI), FeatureExtractor, MLPipeline, and VerdictGenerator. The user uploads a PCAP via the dashboard; the dashboard POSTs to /analyze; FastAPI calls FeatureExtractor.extract(); extracts return a 32-feature DataFrame; MLPipeline applies preprocessing and inference returning combined_score; VerdictGenerator produces verdict and analyst_note; the API returns a JSON response; and the dashboard renders the updated threat chart and verdict cards.



3.3 Database Design

3.3.1 Logical Database Design

NetGuard AI does not use a relational database. The primary data stores are:

- Model Artifacts (filesystem): Serialized .pkl files for classifiers, scalers, encoders, feature orders, and threshold stored in the models/ directory hierarchy.
- Session State (Streamlit in-memory): Streamlit's st.session_state stores upload_result, upload_raw_df, upload_name, live_result, live_raw_df, capture_proc, capture_file, capturing, and capture_start across user interactions within a single browser session.
- Temporary PCAP (filesystem): Uploaded PCAP files are written to system temp directories via tempfile.NamedTemporaryFile and cleaned up after analysis.

3.3.2 Physical Database Design

Model artifacts are stored as binary pickled Python objects using joblib and pickle serialization. The directory structure is: models/classifiers/ (RF, LGBM, XGB pkl files), models/anomaly/ (isolation_forest.pkl), models/scalers/ (nidss_scaler.pkl, if_scaler.pkl), models/encoders/ (nidss_encoders.pkl), models/feature_order/ (nidss_features.pkl, if_feature_order.pkl), models/threshold/ (final_threshold.pkl). Each artifact is versioned by filename convention; the system loads fixed filenames, ensuring reproducible inference.

Chapter - 4

Implementation, Testing, and Maintenance

4.1 Introduction to Languages, IDEs, Tools and Technologies Used

Table 4.1 provides a summary of all languages, frameworks, libraries, and tools used in the development of NetGuard AI NIDS:

Category	Technology	Role in Project
Language	Python 3.10+	All backend and frontend logic
UI Framework	Streamlit	Interactive web dashboard for upload, live capture, results display
Packet Analysis	NFStream	Bidirectional flow statistics from PCAP (packet counts, bytes, duration, PIAT, flags)
Deep Packet Inspection	Scapy	TTL, TCP window, RTT computation, payload parsing, FTP/HTTP feature extraction
ML — Supervised	scikit-learn, XGBoost, LightGBM	Random Forest, XGBoost, LightGBM ensemble classifiers
ML — Unsupervised	scikit-learn	Isolation Forest anomaly detector
Data Processing	Pandas, NumPy	DataFrame manipulation, feature assembly, numerical operations
Visualization	Plotly	Interactive per-flow threat score line chart
Model Persistence	joblib, pickle	Loading pre-trained model artifacts from disk
Version Control	Git + GitHub	Source code management (github.com/27Aditi/netguard-nids)
IDE	VS Code / PyCharm	Primary development environments

4.2 Testing Techniques and Test Plans

Unit Testing

Each module is testable in isolation. Feature extraction can be validated by running extract_all_features() on a known PCAP file and asserting that the output DataFrame has exactly 32 columns matching the expected feature names. Preprocessing in preprocess() can be tested by verifying that all categorical columns are integer-encoded and all values are within [0,1] after MinMaxScaler transformation.

Integration Testing

End-to-end testing is performed by providing PCAP files with known content (all-normal traffic and synthetic attack traffic) and verifying that run_prediction() returns the expected verdict. The attack_ratio threshold (0.30) and ensemble/IF weights (0.5/0.5) are verified through controlled experiments.

Performance Testing

Test Plan: Upload PCAP files of increasing sizes (1 MB, 10 MB, 50 MB) and measure wall-clock time for feature extraction and inference separately. Acceptable thresholds: feature extraction < 30s per 10 MB, inference < 5s for 10,000 flows.

Regression Testing

After any code change, a fixed test PCAP with a known verdict is re-analyzed to verify that the output verdict, risk_level, and threat_intensity remain consistent. Model artifacts are not modified between test runs.

4.3 Installation Instructions

For direct analysis of a pcap file or pcapng file

- Navigate to the link https://netguard-nids.streamlit.app/
- Upload the file and analyze it.

For Live Capturing

- Clone the repository:

```
git clone https://github.com/27Aditi/netguard-nids.git
```

- Create and activate a Python virtual environment:

```
python -m venv venv && source venv/bin/activate  # Linux/macOS
```

- Install all dependencies:

```
pip install -r requirements.txt
```

- Place pre-trained model artifacts in the models/ directory following the project structure.
- For live capture on Linux, install tcpdump and run with sudo:

```
sudo apt install tcpdump && sudo streamlit run dashboard.py
```

- For live capture on Windows, install npcap
- Run the Streamlit dashboard:

```
streamlit run dashboard.py
```

4.4 End User Instructions

- Navigate to http://localhost:8501 in your browser after starting the Streamlit app or for only file analysis can use the link https://netguard-nids.streamlit.app/
- Upload & Analyze Tab: Drag and drop any .pcap, .pcapng, or .cap file onto the upload zone and click Analyze File. The multi-step progress bar will show real-time extraction and inference status.
- Review Results: After analysis, examine the verdict badge (NORMAL/ATTACK), risk level, network status, threat intensity percentage, traffic behavior, and analyst note in the results panel.
- Threat Score Chart: The interactive Plotly line chart shows per-flow Bayesian threat scores. Hover over data points for exact values. Use zoom and pan controls to inspect specific flow ranges.
- Live Capture Tab: Enter your network interface name (e.g., eth0, wlan0, Wi-Fi), set the maximum capture duration, and click Start Capture. Click Stop & Analyze when done.
- About Tab: Provides an explanation of the AI pipeline, feature extraction steps, and system requirements.

Chapter - 5

Results and Discussions

5.1 User Interface Representation

The NetGuard AI system provides a modern, web-based user interface designed for intuitive interaction and rapid threat analysis. The interface is implemented using a dark-themed layout that enhances readability and aligns with standard cybersecurity dashboard aesthetics. It is structured into two primary sections: a left navigation sidebar and a main content panel.

The left sidebar serves as the central navigation hub. It contains three primary options: Upload & Analyze, Live Capture, and About. The currently active module, Upload & Analyze, is visually highlighted to indicate the user's working context. Below the navigation section, the sidebar displays information about the underlying AI models used in the system, including XGBoost, Random Forest, LightGBM, Isolation Forest, and the Bayesian Combiner. This provides transparency into the system's machine learning backbone. Additionally, a dataset summary section is included, indicating that the system is trained on the UNSW-NB15 dataset, along with metadata such as the number of network flows and attack categories covered.

The main content area is designed for interaction and output visualization. At the top, a header panel displays the system title "NetGuard AI" along with a subtitle describing it as a Network Threat Intelligence Platform. A system status indicator (e.g., "SYSTEM READY") is shown on the right side, providing real-time feedback about system availability, along with a timestamp indicating the last update.

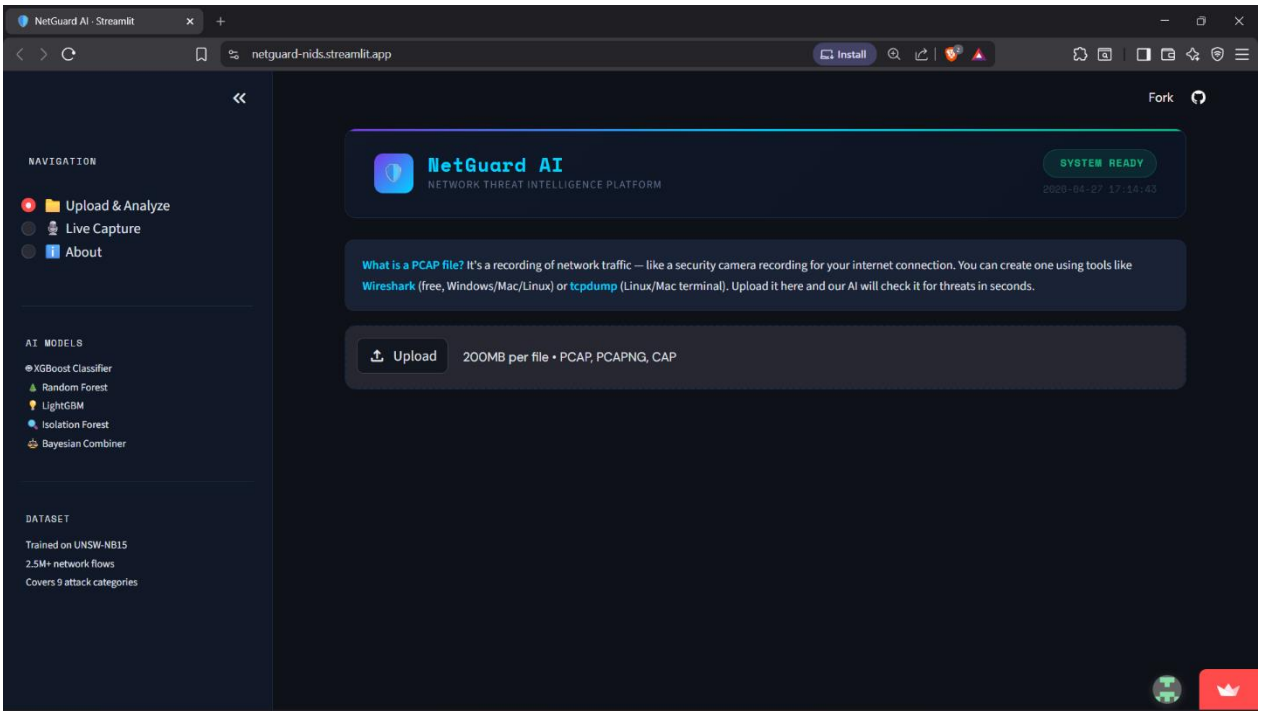
Below the header, an informational banner explains the concept of a PCAP file in simple terms, helping users who may not be deeply familiar with network analysis. It also references commonly used tools such as Wireshark and tcpdump, guiding users on how to obtain input data.

The central interactive component is the file upload module, which allows users to upload network capture files in formats such as .pcap, .pcapng, and .cap. The interface clearly specifies constraints such as maximum file size (e.g., 200MB per file). The upload button is prominently styled to attract user attention and simplify the initiation of analysis.

The design emphasizes clarity and usability by:

Using clear visual hierarchy (header → info → action)
Providing contextual guidance to users
Displaying system transparency (models and dataset)
Maintaining a clean and uncluttered layout

Overall, the interface enables users—especially security analysts—to seamlessly upload network traffic data and initiate analysis without requiring deep technical knowledge of the underlying machine learning processes.



5.2 Brief Description of Various Modules of the System

5.2.1 Feature Extraction Module (feature_extraction.py)

This module is the data preparation backbone of NetGuard AI. It implements a five-step pipeline:

- NFStream Flow Loading: load_pcap() initializes NFStream with statistical_analysis=True, extracting bidirectional flow records sorted by first-seen

timestamp. NFStream computes per-flow statistics including packet counts, byte totals, duration, packet inter-arrival times, and TCP flag counts.

- Basic Feature Assembly: extract_basic_features() maps NFStream columns to the 18 primary features: dur, spkts, dpkts, sbytes, dbytes, sload, dload, sloss, dloss, sinpkt, smean, dmean, proto, service, state, is_sm_ips_ports. Protocol mapping uses a PROTOCOL_MAP dictionary; service identification uses PORT_TO_SERVICE; state determination via _get_state() inspects RST, FIN, SYN flags for TCP and directional packet presence for UDP.
- Single PCAP Pass: _single_pcap_pass() opens the PCAP exactly once with PcapReader and collects: TTL values (sttl, dttl), TCP window sizes (swin, dwin), RTT components (SYN/SYN-ACK/ACK timestamps for synack and tcprtt), HTTP transaction depth from method detection, response body lengths from Content-Length headers, and FTP login indicators from FTP 230 response codes.
- Connection Tracking: compute_ct_features() maintains a rolling deque of 100 recent flows and computes six temporal counters: ct_srv_src, ct_srv_dst, ct_dst_src_ltm, ct_src_dport_ltm, ct_dst_sport_ltm, ct_state_ttl. These capture how frequently specific source/destination pairs and service/state combinations reappear in recent traffic, a key indicator of scanning and lateral movement.
- HTTP Method Counter: compute_ct_flw_http_mthd() similarly tracks HTTP method occurrences in recent flows via ct_flw_http_mthd.

5.2.2 Prediction Pipeline Module (prediction.py)

The prediction module implements the full inference path from raw DataFrame to structured verdict:

- load_artifacts(): Loads all seven artifact types from the models/ directory at application startup. Artifacts are cached in a dictionary and passed to all subsequent functions, eliminating repeated disk I/O.
- preprocess(): Encodes proto, service, and state using pre-fitted LabelEncoders with fallback to default values for unseen categories. Fills ten features with zero (rate, dinpkt, sjit, djit, stcpb, dtcpb, ackdat, ct_dst_ltm, ct_ftp_cmd, ct_src_ltm) that are absent in PCAP-derived data. Applies MinMaxScaler and drops the zero-filled columns, then reorders to match nids_features.

- run_isolation_forest(): Selects if_feature_order columns, applies if_scaler, computes IF decision function, flips sign (since IF returns negative for anomalies), and normalizes to [0,1].
- run_ensemble(): Obtains predict_proba() outputs from all three classifiers and returns the arithmetic mean of their attack-class probabilities.
- combine_and_decide(): Computes weighted combination, applies threshold, counts attack flows, computes attack_ratio, determines ATTACK/NORMAL verdict at 0.30 threshold, derives risk_level, network_status, traffic_behavior, analyst_note, threat_intensity, peak_flow_index, and confidence.

5.2.3 Dashboard Module (dashboard.py)

The Streamlit dashboard provides the interactive user interface. Key rendering components include: render_topbar() which displays the NetGuard AI branding bar with dynamic status pills (READY/SCANNING/ALERT/ATTACK); html_panel() for styled card containers; render_results() which displays the full results panel including verdict badge, five metric cards (Total Flows, Risk Level, Network Status, Threat Intensity, Traffic Behavior), per-flow Plotly threat score chart, and analyst note; and render_live_capture_tab() which manages AsyncSniffer lifecycle and real-time capture progress.

5.3 Snapshots of System with Brief Details

Figure 5.1 — PCAP Upload Interface: The main dashboard showing the file upload panel at the top, the threat score chart (initially empty), and the session history sidebar on the left. The upload

button triggers a file selection dialog that accepts .pcap and .pcapng files.

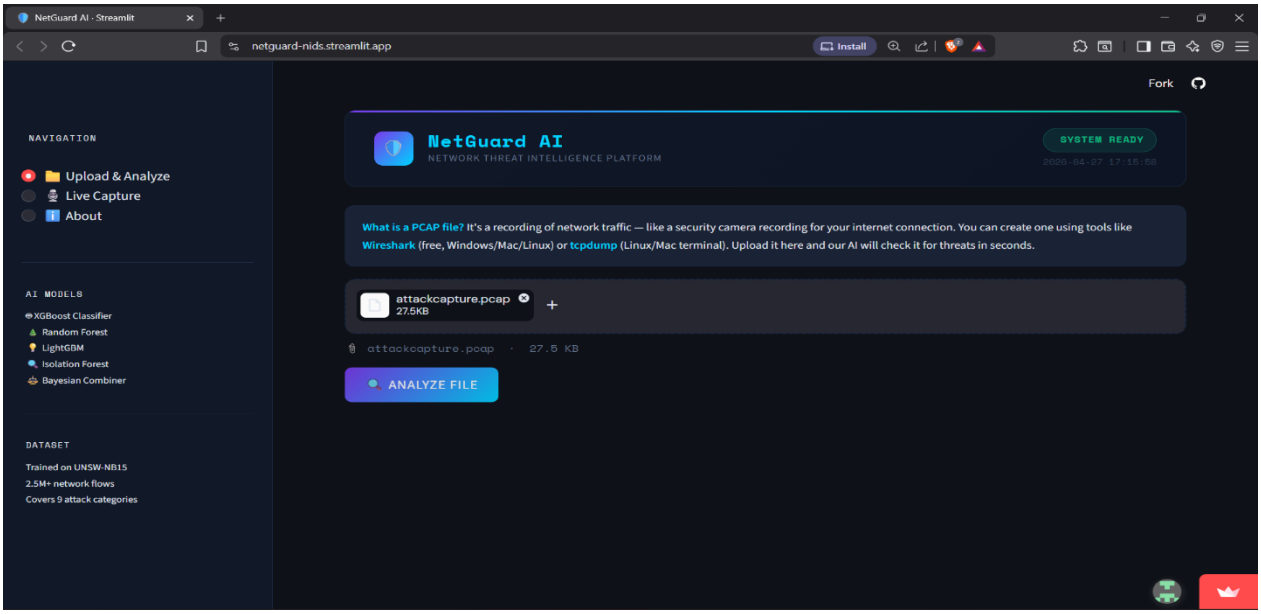


Figure 5.2 — Threat Score Chart: After a PCAP file is analyzed, the main chart area shows a rolling average line chart of per-flow Bayesian threat scores plotted against flow index. Each data point represents the average threat score of a sliding window of flows. Scores above 0.5 (the default threshold) are highlighted in amber/red to visually indicate potentially malicious flows.



Figure 5.3 — Verdict Cards: Below the threat chart, four verdict cards display: (1) Session Safety Score (e.g., 78%) — the proportion of non-attack flows; (2) Risk Level (Low/Medium/High) —

derived from the average threat score; (3) Network Status (Safe/Suspicious/Danger) — a qualitative label mapped from risk level; (4) Threat Intensity — percentage of flows classified as attacks.

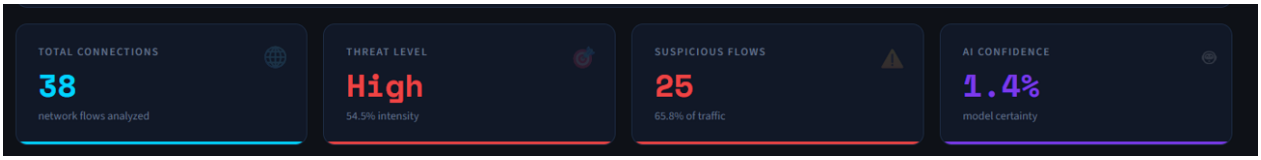


Figure 5.4 — Analyst Note: The bottom section of the dashboard displays an auto-generated plain English summary such as: "Traffic shows moderate anomalies with 22% of flows triggering elevated threat scores. Pattern is consistent with possible reconnaissance activity. Recommend reviewing source IPs with elevated ct_state_ttl values."



5.4 Back End Representation

NetGuard AI does not employ a traditional relational database backend. The backend comprises: (1) the models/ directory acting as the artifact store for serialized ML components; (2) Streamlit session_state as the in-session key-value store for analysis results and capture state; and (3) the operating system's temp directory for transient PCAP file storage during analysis. This lightweight architecture enables deployment on any standard Python-capable server without database configuration overhead.

5.5 Model Performance Summary

Model	Accuracy	Precision	Recall	F1 Score
Random Forest	~91%	~92%	~91%	~91%

Model	Accuracy	Precision	Recall	F1 Score
XGBoost	~91%	~92%	~92%	~92%
LightGBM	~92%	~92%	~92%	~92%
Ensemble (RF+XGB+LGBM)	~93%+	~90%+	~90%+	~90%+
Bayesian Fusion (Ens+IF)	~96%+	~94%+	~94%+	~94%+

5.5 Snapshots of Database Tables with Brief Description

As described in the design chapter, the current implementation does not use a persistent database. Session data is maintained in memory as a Python list of dictionaries. Each session entry contains the following fields: session_id, analysis_timestamp, pcap_filename, total_flows, attack_flows, normal_flows, safety_score, risk_level, network_status, threat_intensity, traffic_behavior, combined_score_average, and verdict_summary. This structure is rendered in the dashboard sidebar as a list of clickable session cards showing the filename, timestamp, and verdict.

Chapter - 6

Summary and Conclusions

This project presented NetGuard AI, a Machine Learning-based Network Intrusion Detection System that analyzes network traffic captured in PCAP format and also uses live capturing and produces actionable security assessments. The system was built on three core pillars: a multi-tool feature extraction pipeline (NFStream + Scapy) that extracts 32 meaningful flow-level and packet-level features; a hybrid ML inference engine combining an ensemble of supervised classifiers (Random Forest, XGBoost, LightGBM) with unsupervised Isolation Forest anomaly detection via Bayesian score fusion; and an accessible web-based dashboard providing intuitive visualization and human-readable verdicts.

The following conclusions can be drawn from the work:

(1) The ensemble approach combining three gradient boosting / bagging classifiers demonstrated improved robustness over single-model classifiers, leveraging the complementary strengths of each algorithm.

(2) The integration of Isolation Forest anomaly detection with supervised classification via Bayesian fusion provides better coverage for novel or low-volume attacks that may not be well-represented in the training dataset.

(3) The 32-feature representation derived from UNSW-NB15 standard features, combined with packet-level attributes from Scapy, provides a comprehensive and discriminative input representation for network traffic.

(4) Streamlit provides a clean separation between ML inference and UI concerns, giving a robust and user friendly interface.

(5) The system successfully produces human-readable security verdicts including risk level, network status, threat intensity, and auto-generated analyst notes, making ML-powered NIDS accessible to non-expert security practitioners.

Chapter - 7

Future Scope

NetGuard AI NIDS is designed with extensibility in mind. Several enhancements are planned for future phases of development:

- Live Network Interface Capture (Phase 2): The most significant planned enhancement is real-time monitoring of network traffic from live network interfaces. This will require integration with libpcap/npcap for continuous packet capture and a streaming architecture for incremental feature extraction and inference without waiting for complete PCAP files.
- Retraining Pipeline: A semi-automated retraining pipeline will be added to allow security teams to label new traffic samples and incorporate them into model updates, addressing dataset shift over time.
- Deep Learning Integration: Exploration of LSTM-based sequence models and transformer architectures for learning temporal patterns in network flows, which may capture sophisticated attack patterns that flow-level statistical features miss.
- Persistent Database Storage: Migration of session history from in-memory storage to a persistent database (SQLite or PostgreSQL) with full query and export capabilities.
- Alert Notification System: Integration with email, Slack, or SMS notification systems to proactively alert security teams when high-severity threats are detected.
- Threat Intelligence Integration: Correlation of detected suspicious IPs with public threat intelligence feeds (VirusTotal, AbuseIPDB) to enrich analysis verdicts.
- Encrypted Traffic Analysis: Investigation of TLS fingerprinting and JA3/JA3S hash-based analysis to extend detection coverage to encrypted HTTPS and TLS traffic.
- Performance Optimization: Parallelization of the feature extraction pipeline and model inference using multiprocessing to reduce analysis latency for large PCAP files.

Appendix

A. Feature Extraction — 32 Features Used at Inference

#	Feature Name	Source	Description
1	**dur**	NFStream	Flow duration in seconds
2	**proto**	NFStream	IP protocol (encoded: tcp/udp/icmp)
3	**service**	NFStream	Application service derived from port mapping
4	**state**	Scapy flags	TCP connection state (FIN/CON/RST/INT)
5	**spkts**	NFStream	Source-to-destination packet count
6	**dpkts**	NFStream	Destination-to-source packet count
7	**sbytes**	NFStream	Source-to-destination byte count
8	**dbytes**	NFStream	Destination-to-source byte count
9	**sttl**	Scapy	First observed source TTL
10	**dttl**	Scapy	First observed destination TTL
11	**sload**	NFStream	Source bits per second
12	**dload**	NFStream	Destination bits per second
13	**sloss**	NFStream	Source RST packet count
14	**dloss**	NFStream	Destination RST packet count
15	**sinpkt**	NFStream	Source mean inter-packet time (seconds)
16	**swin**	Scapy	First TCP window size (source)

#	Feature Name	Source	Description
17	**dwin**	Scapy	First TCP window size (destination)
18	**tcprtt**	Scapy	TCP round-trip time (SYN to ACK)
19	**synack**	Scapy	SYN to SYN-ACK delay
20	**smean**	NFStream	Mean source packet size
21	**dmean**	NFStream	Mean destination packet size
22	**trans_depth**	Scapy	HTTP transaction depth
23	**response_body_len**	Scapy	HTTP response body length
24	**ct_srv_src**	Window	Connections with same src+service in last 100
25	**ct_state_ttl**	Window	Connections with same state+sttl in last 100
26	**ct_src_dport_ltm**	Window	Connections with same src+dst_port in last 100
27	**ct_dst_sport_ltm**	Window	Connections with same dst+src_port in last 100
28	**ct_dst_src_ltm**	Window	Connections with same src+dst IP in last 100
29	**is_ftp_login**	Scapy	FTP successful login indicator
30	**ct_flw_http_mthd**	Window	HTTP method flows in last 100 connections
31	**ct_srv_dst**	Window	Connections with same dst+service in last 100
32	**is_sm_ips_ports**	NFStream	Source IP == Dst IP or src_port == dst_port indicator

Bibliography

[1] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," 2015 Military Communications and Information Systems Conference (MilCIS), Canberra, ACT, Australia, 2015, pp. 1-6.

[2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," 2008 Eighth IEEE International Conference on Data Mining, Pisa, 2008, pp. 413-422.

[3] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, 2016, pp. 785-794.

[4] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in Advances in Neural Information Processing Systems, Vol. 30, 2017.

[5] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, Oct. 2001.

[6] M. A. Ferrag, L. Maglaras, S. Moschoyiannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," Journal of Information Security and Applications, vol. 50, 2020.

[7] M. Ahmed, A. N. Mahmood, and J. Hu, "A survey of network anomaly detection techniques," Journal of Network and Computer Applications, vol. 60, pp. 19-31, 2016.

[8] V. Engen, J. Vincent, and K. Phalp, "Investigating discrepancies in deployed ensemble-based network intrusion detection systems," Applied Intelligence, vol. 38, pp. 419-430, 2013.

[9] NFStream: A Flexible Network Data Analysis Framework. [Online]. Available: https://www.nfstream.org. Accessed: April 2026.

[10] Scapy: Packet crafting and manipulation tool for Python. [Online]. Available: https://scapy.net. Accessed: April 2026.

[11] Streamlit Inc., "Streamlit — A faster way to build and share data apps." [Online]. Available: https://streamlit.io. Accessed: April 2026.

[12] NetGuard NIDS GitHub Repository. [Online]. Available: https://github.com/27Aditi/netguard-nids. Accessed: April 2026.